

1 Синтаксис грамматики

$$\begin{aligned}\langle Program \rangle &::= \langle Item \rangle \mid \langle Item \rangle \langle ItemSeparator \rangle \langle Program \rangle \\ \langle ItemSeparator \rangle &::= \text{NEWLINE} \\ \langle Item \rangle &::= \langle Definition \rangle \mid \langle Expr \rangle \\ \langle Definition \rangle &::= \text{let } \langle Identifier \rangle = \langle Expr \rangle \\ \langle Expr \rangle &::= \langle Abstraction \rangle \mid \langle Argument \rangle \mid \langle Argument \rangle \langle Arguments \rangle \\ \langle Abstraction \rangle &::= \backslash \langle Identifiers \rangle . \langle Expr \rangle \\ \langle Identifiers \rangle &::= \langle Identifier \rangle \mid \langle Identifier \rangle \langle Identifiers \rangle \\ \langle Arguments \rangle &::= \langle ApplicationArgument \rangle \mid \langle ApplicationArgument \rangle \langle Arguments \rangle \\ \langle ApplicationArgument \rangle &::= \langle Argument \rangle \mid \langle Abstraction \rangle \\ \langle Argument \rangle &::= \langle Identifier \rangle \mid (\langle Expr \rangle)\end{aligned}$$

Имя переменной $\langle Identifier \rangle$ задается следующими правилами:

$$\begin{aligned}\langle Identifier \rangle &::= \langle Letter \rangle \mid \langle Letter \rangle \langle IdentifierTail \rangle \\ \langle IdentifierTail \rangle &::= \langle LetterOrDigit \rangle \mid \langle LetterOrDigit \rangle \langle IdentifierTail \rangle \\ \langle LetterOrDigit \rangle &::= \langle Letter \rangle \mid \langle Digit \rangle\end{aligned}$$

где $\langle Letter \rangle$ — латинская буква в любом регистре, $\langle Digit \rangle$ — цифра. Содержательно: имя начинается с латинской буквы и состоит далее из латинских букв и цифр. Ключевое слово `let` зарезервировано и не может быть именем переменной.

Пробелы и табуляции между лексемами незначимы. Терминал `NEWLINE` обозначает конец строки; несколько переводов строк подряд (пустые строки) допускаются и трактуются как один разделитель. Элементы программы (определения и выражения) записываются каждый на своей строке; каждое определение занимает ровно одну строку, и на строке не может быть более одного определения.

2 Свойства грамматики

1. **Отсутствие эпсилон-переходов.** Правая часть каждого правила не пуста: ни одна альтернатива не является ϵ . Последовательности задаются только правилами факторизованного вида $X ::= A \mid A B$, поэтому пустой список ни в одном месте не выводим, а пустое слово не выводится ни из одного нетерминала.
2. **Программа не может быть пустой.** Поскольку $\langle Program \rangle$ требует хотя бы один $\langle Item \rangle$, а $\langle Item \rangle$ не пуст, пустая программа не выводима.
3. **Определения необязательны.** $\langle Item \rangle ::= \langle Definition \rangle \mid \langle Expr \rangle$, поэтому программа может состоять из одних выражений, одних определений или их произвольного чередования, например:

```

let I = \x.x
I y
let t = a b c

```

Вычисляется последнее выражение программы; каждое определение действует на все последующие выражения.

4. **Однозначность.** Каждое выражение имеет единственное дерево вывода: правила записаны в факторизованной форме $X ::= A \mid A B$ без левой рекурсии и эpsilon-переходов, а выбор альтернативы определяется первым символом и наличием последующих аргументов, без просмотра всего входа: **let** однозначно начинает определение, $\backslash$ — абстракцию, а атом, за которым следует хотя бы один аргумент, однозначно порождает ветвь $\langle Argument \rangle \langle Arguments \rangle$, без аргументов — только $\langle Argument \rangle$. Резервированное слово **let** не может встретиться в качестве имени.
5. **Абстракция как аргумент.** Аппликация может заканчиваться лямбда-абстракцией без скобок $\backslash x \ y. \ y \ x \ \backslash x.x$, что эквивалентно $\backslash x \ y. \ ((y \ x) \ (\backslash x.x))$; это обеспечивает правило $\langle ApplicationArgument \rangle ::= \langle Argument \rangle \mid \langle Abstraction \rangle$.
6. **Отсутствие левой рекурсии и факторизация.** Ни в одном правиле нет левой рекурсии. Каждая конструкция записана в форме $X ::= A \mid A B$: в $\langle Expr \rangle ::= \langle Abstraction \rangle \mid \langle Argument \rangle \mid \langle Argument \rangle \langle Arguments \rangle$ вторая и третья ветви имеют общий префикс $\langle Argument \rangle$ и различаются наличием непустого списка $\langle Arguments \rangle$, поэтому аппликация и одиночный аргумент различимы: за атомом следует хотя бы один аргумент — третья ветвь, иначе — вторая.
7. **Факторизация для перечислений.** Списки $\langle Identifiers \rangle$ и $\langle Arguments \rangle$ записаны единообразно в форме $X ::= A \mid A B$ и различаются только типом элемента: $\langle Identifier \rangle$ и $\langle ApplicationArgument \rangle$ соответственно.

3 Примеры вывода

3.1 Пример 1: Определение $\text{let } K = \backslash x \ y. \ x$

- $\langle Program \rangle \Rightarrow \langle Item \rangle$
- $\Rightarrow \langle Definition \rangle$
- $\Rightarrow \text{let } \langle Identifier \rangle = \langle Expr \rangle$
- $\Rightarrow \text{let } K = \langle Expr \rangle$

- $\Rightarrow \text{let } K = \langle Abstraction \rangle$
- $\Rightarrow \text{let } K = \setminus \langle Identifiers \rangle . \langle Expr \rangle$
- $\Rightarrow \text{let } K = \setminus \langle Identifier \rangle \langle Identifiers \rangle . \langle Expr \rangle$
- $\Rightarrow \text{let } K = \setminus x \langle Identifier \rangle . \langle Expr \rangle$
- $\Rightarrow \text{let } K = \setminus x y . \langle Expr \rangle$
- $\Rightarrow \text{let } K = \setminus x y . \langle Argument \rangle$
- $\Rightarrow \text{let } K = \setminus x y . \langle Identifier \rangle$
- $\Rightarrow \text{let } K = \setminus x y . x$

3.2 Пример 2: Аппликация, заканчивающаяся абстракцией $\setminus x y . y x \setminus x . x$

- $\langle Expr \rangle \Rightarrow \langle Abstraction \rangle$
- $\Rightarrow \setminus \langle Identifiers \rangle . \langle Expr \rangle$
- $\Rightarrow \setminus \langle Identifier \rangle \langle Identifiers \rangle . \langle Expr \rangle$
- $\Rightarrow \setminus x \langle Identifier \rangle . \langle Expr \rangle$
- $\Rightarrow \setminus x y . \langle Expr \rangle$
- $\Rightarrow \setminus x y . \langle Argument \rangle \langle Arguments \rangle$
- $\Rightarrow \setminus x y . y \langle Arguments \rangle$
- $\Rightarrow \setminus x y . y \langle ApplicationArgument \rangle \langle Arguments \rangle$
- $\Rightarrow \setminus x y . y x \langle Arguments \rangle$
- $\Rightarrow \setminus x y . y x \langle ApplicationArgument \rangle$
- $\Rightarrow \setminus x y . y x \langle Abstraction \rangle$
- $\Rightarrow \setminus x y . y x \setminus \langle Identifiers \rangle . \langle Expr \rangle$
- $\Rightarrow \setminus x y . y x \setminus x . \langle Expr \rangle$
- $\Rightarrow \setminus x y . y x \setminus x . \langle Argument \rangle$
- $\Rightarrow \setminus x y . y x \setminus x . \langle Identifier \rangle$
- $\Rightarrow \setminus x y . y x \setminus x . x$

3.3 Пример 3: Аппликация S K K

- $\langle Expr \rangle \Rightarrow \langle Argument \rangle \langle Arguments \rangle$
- $\Rightarrow \langle Identifier \rangle \langle Arguments \rangle$
- $\Rightarrow S \langle Arguments \rangle$
- $\Rightarrow S \langle ApplicationArgument \rangle \langle Arguments \rangle$
- $\Rightarrow S K \langle Arguments \rangle$
- $\Rightarrow S K \langle ApplicationArgument \rangle$
- $\Rightarrow S K K$